DAY 39

Deploying Machine Learning with a Flask REST API

60-Day Data Science Learning Journey

Goal Build a Flask API that receives JSON, validates input, calls a saved Scikit-learn pipeline, and returns a prediction.
Endpoints `GET /health` and `POST /predict`
Prerequisite A compatible `student_result_model.joblib` from the Day 38 project, using study_hours and attendance.
Responsible use Synthetic student data only. Not suitable for real academic decisions.

Concepts, code walkthrough, request examples, test cases, common mistakes, and revision tasks.

1. Learning objectives

• Explain API, REST endpoint, HTTP, and JSON.

• Understand GET versus POST.

• Build Flask endpoints for health checks and predictions.

• Connect a saved ML pipeline to a request handler.

• Validate input and return appropriate HTTP status codes.

• Recognize basic security and production limitations.

2. How the API workflow works

A client sends an HTTP request. Flask receives it, checks the input, calls the saved model, and returns a JSON response. The API is the bridge between the client application and the model.

Stage	Purpose
Client	Sends study_hours and attendance in JSON.
Flask endpoint	Checks required fields, numeric types, and ranges.
Saved pipeline	Applies preprocessing and predicts a label.
JSON response	Returns prediction and optional class probability estimates.

3. HTTP methods and status codes

GET commonly retrieves information. **POST** commonly sends data to the server, such as input for a prediction.

• **200 OK:** request succeeded.

• **400 Bad Request:** client sent invalid input.

• **404 Not Found:** endpoint does not exist.

• **405 Method Not Allowed:** HTTP method is not supported on that endpoint.

• **500 Internal Server Error:** server could not complete the operation.

Example request JSON

```
{
  "study_hours": 6,
  "attendance": 80
}
```

Example response shape (illustrative)

```
{
  "prediction": "Pass",
  "message": "Prediction generated successfully.",
  "educational_only": true
}
```

The actual prediction depends on the fitted model. A model output is not a guarantee of real-world performance.

4. Flask basics

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.get("/health")
def health():
    return jsonify({"status": "ok"}), 200

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

`Flask(__name__)` creates the application. The route decorator maps an HTTP method and path to a Python function. `jsonify()` returns JSON. The built-in server is intended for development, not public production hosting.

5. Complete API project

The separate Python file contains the complete commented implementation. Its main parts are:

• Loads a trusted model once when the application starts.

• Defines `GET /health` and `POST /predict`.

• Rejects invalid JSON, missing fields, booleans, non-finite numbers, and out-of-range values.

• Creates a DataFrame with the exact feature names expected by the saved pipeline.

• Returns structured JSON errors and optionally includes class probability estimates.

• Runs on localhost with debug mode disabled for local practice.

Core prediction pattern

```
data = request.get_json(silent=True)
if not isinstance(data, dict):
    return jsonify({"error": "Send a valid JSON object."}), 400

features = pd.DataFrame([{
    "study_hours": float(data["study_hours"]),
    "attendance": float(data["attendance"])
}])
prediction = model.predict(features)[0]
return jsonify({"prediction": str(prediction)}), 200
```

The complete script includes the required-field checks and input-range validation omitted from this short illustrative excerpt. Run the Day 38 training project first to create the model file. Only load serialized files from trusted sources.

6. API test cases

Case	Example	Expected result
Valid input	Both numeric fields provided	200 with prediction
Missing field	Only study_hours	400 and missing_fields
Wrong type	Attendance = 'abc'	400 validation error
Out of range	Attendance = 140	400 validation error
Unknown path	/unknown	404 Not Found
Wrong method	GET /predict	405 Method Not Allowed

7. Security and responsible deployment

• Never enable Flask debug mode on a public production server.

• Use authentication and authorization when an endpoint handles private or sensitive functionality.

• Consider request limits, rate limiting, monitoring, and careful logging for real services.

• Avoid returning stack traces or internal paths to clients.

• Use a production WSGI server and secure hosting for public deployment.

• Track dependency and model versions, and validate every input.

• This synthetic student model is educational only and must not drive real academic decisions.

8. Common mistakes

Model file missing: Create it with the Day 38 training script and check the working directory.

Feature mismatch: Use the same feature names and compatible types as model training.

No validation: Reject missing fields, invalid types, NaN/infinity, and unreasonable ranges.

Probability treated as certainty: Model probabilities are estimates, not guarantees.

Debug mode public: Disable debug mode and use proper production hosting.

No tests: Test successful requests and multiple invalid inputs.

9. Practice tasks and suggested answers

Task 1: What is the difference between GET /health and POST /predict?

Answer: Health checks whether the API responds. Prediction accepts data and invokes the model.

Task 2: Why use a DataFrame with named columns?

Answer: The pipeline expects the same feature schema used during training.

Task 3: What should happen if attendance is 140?

Answer: The API rejects it with a 400 response because the example expects a value from 0 to 100.

Task 4: Why disable debug mode in production?

Answer: Debug tools can expose sensitive details and are unsafe for public production use.

Task 5: Name two real deployment requirements.

Answer: Authentication, secure hosting, monitoring, rate limiting, dependency management, and privacy controls are examples.

10. Revision checklist

■ I can explain API, REST, HTTP, and JSON.

■ I understand GET versus POST.

■ I know what Flask route decorators do.

■ I understand how the saved model is called by an endpoint.

■ I can explain input validation and common HTTP status codes.

■ I know why a local demonstration is not the same as production deployment.

LinkedIn post

The separate Day-39-LinkedIn-Post.txt file contains the complete copy-ready post and the requested GitHub repository link:

https://github.com/DeveloperDesigner-star/Datasciencepdfs

Verification note

The Python file was syntax-checked. The Flask application was not runtime-tested during file generation. It requires installed dependencies and the compatible Day 38 model file.